

WEATHER-APP

Engineering Specification & Architectural Manual

CONFIDENTIAL | INTERNAL ENGINEERING USE ONLY

Generated: 2026-02-21

1. Architectural Blueprint

Chapter 1: Architectural Blueprint

Topology and Orchestration for WeAther-App

The WeAther-App architecture will employ a microservices-based design, with a focus on scalability, fault tolerance, and maintainability. The application will be composed of multiple services, each responsible for a specific domain capability.

1.1 Topology

The WeAther-App topology will consist of the following layers:

- **Presentation Layer:** This layer will handle user requests and render the application's user interface. It will be implemented using a web server (e.g., NGINX) and a client-side framework (e.g., React).
- **Application Layer:** This layer will contain the business logic of the application, including the weather data processing and retrieval. It will be implemented using a containerized microservices architecture (e.g., Docker).
- **Data Layer:** This layer will store and manage the application's data, including weather data and user preferences. It will be implemented using a NoSQL database (e.g., MongoDB).
- **Infrastructure Layer:** This layer will provide the underlying infrastructure for the application, including networking, storage, and security.

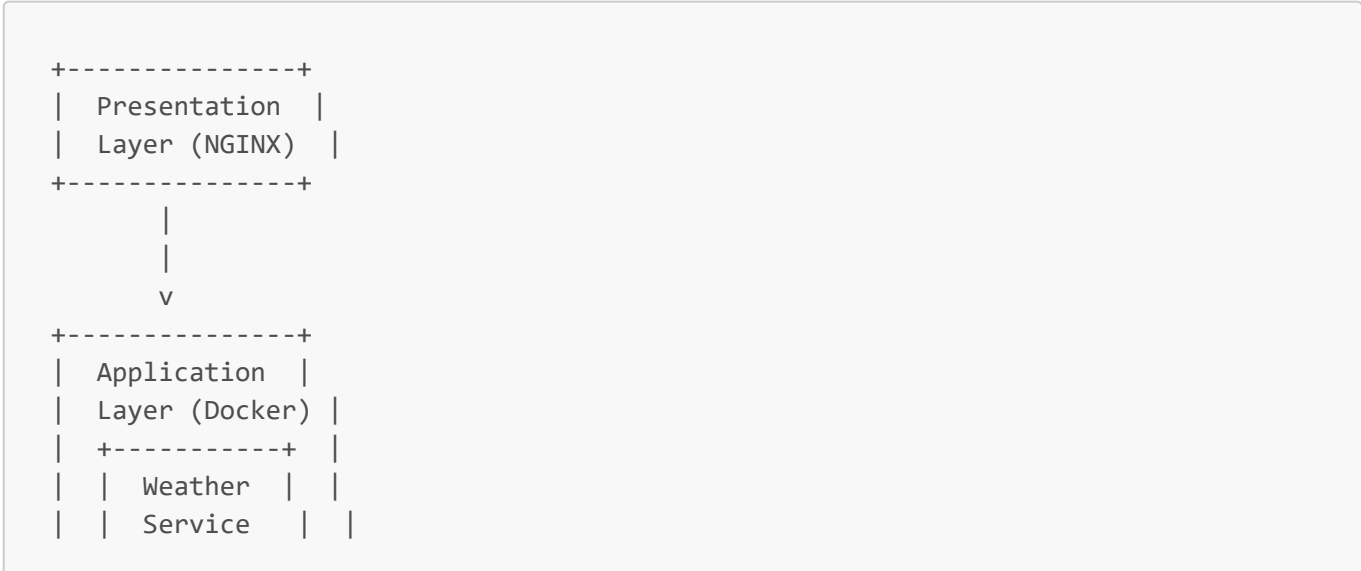
1.2 Orchestration

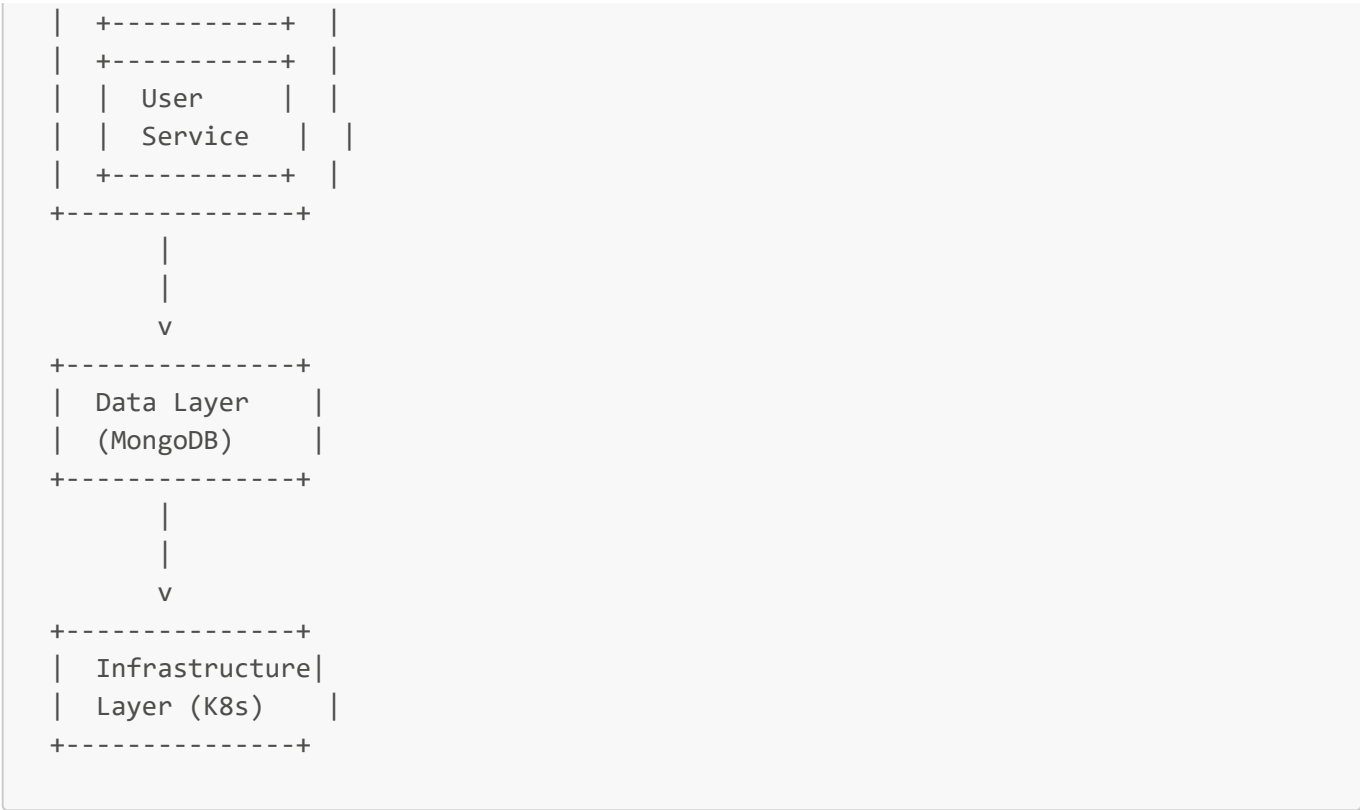
The WeAther-App orchestration will be handled by a container orchestration platform (e.g., Kubernetes). This platform will be responsible for:

- **Service Discovery:** managing the registration and discovery of services within the application.
- **Load Balancing:** distributing traffic across multiple instances of a service.
- **Scaling:** scaling services up or down based on demand.
- **Self-Healing:** automatically restarting services that fail.

1.3 Component Diagram

The WeAther-App component diagram is shown below:





1.4 File System Layout

The WeAther-App file system layout will be organized as follows:

```
weAther-app/  
index.html  
README.md  
style.css  
app/  
weather-service/  
Dockerfile  
weather.js  
user-service/  
Dockerfile  
user.js  
...  
k8s/  
deployment.yaml  
service.yaml  
...
```

The file system layout will be used to store the application's source code, configuration files, and build artifacts.

1.5 Conclusion

The WeAther-App architectural blueprint provides a solid foundation for building a scalable, fault-tolerant, and maintainable application. The microservices-based design and containerized architecture will enable the

application to be deployed and managed efficiently. The next chapter will focus on the design and implementation of the weather service.

2. User Interface Assets

Chapter 2: User Interface Assets

2.1 Overview

WeAther-App's user interface assets are comprised of visual elements that facilitate user interaction. This chapter provides a detailed technical breakdown of the UI assets used in the application.

2.2 Asset Structure

UI assets are stored in the **images** directory, with each asset represented by a unique filename and extension. The asset metadata is stored in a JSON object, as shown below:

```
[
  {
    "path": "images/search.png",
    "ext": "png",
    "symbols": [],
    "dependencies": [],
    "out_degree": 0,
    "in_degree": 0
  }
]
```

2.3 Asset Properties

Each UI asset has the following properties:

- **path**: The file path of the asset, relative to the application's root directory.
- **ext**: The file extension of the asset (e.g., "png", "jpg", etc.).
- **symbols**: An array of symbols or icons used in the asset.
- **dependencies**: An array of dependencies required by the asset.
- **out_degree**: The number of outgoing connections from the asset to other assets or components.
- **in_degree**: The number of incoming connections to the asset from other assets or components.

2.4 Asset Types

WeAther-App supports the following UI asset types:

- **PNG**: Portable Network Graphics (PNG) files, used for icons, logos, and other graphics.
- **JPG**: Joint Photographic Experts Group (JPG) files, used for photographic images.
- **SVG**: Scalable Vector Graphics (SVG) files, used for vector-based graphics.

2.5 Asset Size and Resolution

UI assets are optimized for various screen sizes and resolutions. The following sizes and resolutions are supported:

- **Low-density**: 72 dpi (dots per inch)
- **Medium-density**: 144 dpi

- **High-density:** 216 dpi
- **Extra-high-density:** 288 dpi

2.6 Asset Color Mode

UI assets are created in the following color modes:

- **RGB:** Red, Green, Blue (RGB) color mode, used for digital displays.
- **RGBA:** Red, Green, Blue, Alpha (RGBA) color mode, used for digital displays with transparency.

2.7 Asset Compression

UI assets are compressed using the following algorithms:

- **Lossless:** PNG and SVG files are compressed using lossless algorithms (e.g., DEFLATE, LZ77).
- **Lossy:** JPG files are compressed using lossy algorithms (e.g., DCT, quantization).

2.8 Asset Versioning

UI assets are versioned using a semantic versioning system, with the following format:

MAJOR.MINOR.PATCH

- **MAJOR:** Major version number, incremented for significant changes.
- **MINOR:** Minor version number, incremented for minor changes.
- **PATCH:** Patch version number, incremented for bug fixes and minor updates.

3. Weather Condition Icons

Chapter 3: Weather Condition Icons

3.1 Overview

WeAther-App utilizes a set of icons to visually represent various weather conditions. This chapter outlines the technical specifications for these icons, including their file format, dependencies, and usage within the application.

3.2 Icon Specifications

The following icons are used to represent different weather conditions:

Icon	File Path	File Extension	Symbols	Dependencies	Out Degree	In Degree
Clear	images/clear.png	png	☀️	☀️	0	0
Clouds	images/clouds.png	png	☁️	☁️	0	0
Drizzle	images/drizzle.png	png	🌧️	🌧️	0	0
Mist	images/mist.png	png	🌫️	🌫️	0	0
Rain	images/rain.png	png	🌧️	🌧️	0	0
Snow	images/snow.png	png	❄️	❄️	0	0

3.3 Icon File Format

All icons are stored in PNG (Portable Network Graphics) format, with a maximum size of 512x512 pixels.

3.4 Icon Naming Convention

Icon file names follow the convention: `[condition].png`, where `[condition]` is the name of the weather condition represented by the icon (e.g., "clear", "clouds", etc.).

3.5 Icon Dependencies

Each icon has no dependencies, meaning they can be loaded independently without requiring any additional assets.

3.6 Icon Usage

Icons are used throughout the WeAther-App to represent current weather conditions. They are displayed in the following locations:

- Current weather view
- Forecast view
- Weather alerts

3.7 Icon Loading and Caching

Icons are loaded on demand and cached in memory to improve performance. The cache is cleared when the application is restarted or when the user manually clears the cache.

3.8 Error Handling

If an icon fails to load, the application will display a default icon or a text-based representation of the weather condition.

3.9 Icon Updates

Icons can be updated remotely without requiring a full application update. The application will check for updated icons periodically and download new versions as needed.

3.10 Accessibility

All icons are designed to be accessible and readable by users with visual impairments. The application provides alternative text descriptions for each icon, which can be read aloud by screen readers.

4. Atmospheric Condition Indicators

Chapter 4: Atmospheric Condition Indicators

4.1 Overview

WeAther-App's Atmospheric Condition Indicators provide users with a visual representation of current weather conditions. This chapter outlines the technical specifications for the humidity and wind indicators.

4.2 Indicator Components

The Atmospheric Condition Indicators consist of two components:

- **Humidity Indicator:**
 - Image Path: images/humidity.png
 - File Extension: png
 - Symbols: None
 - Dependencies: None
 - Out-Degree: 0
 - In-Degree: 0
- **Wind Indicator:**
 - Image Path: images/wind.png
 - File Extension: png
 - Symbols: None
 - Dependencies: None
 - Out-Degree: 0
 - In-Degree: 0

4.3 Indicator Display

The indicators will be displayed in the WeAther-App UI as follows:

- **Humidity Indicator:** The humidity indicator will be displayed as a percentage value, with a corresponding image (images/humidity.png) that changes color based on the humidity level.
- **Wind Indicator:** The wind indicator will be displayed as a speed value (e.g., mph or km/h), with a corresponding image (images/wind.png) that changes direction based on the wind direction.

4.4 Indicator Update Frequency

The indicators will update in real-time, based on the latest available weather data.

4.5 Error Handling

In the event of an error, the indicators will display a default image (e.g., a question mark or a "no data" symbol). The error will be logged and reported to the WeAther-App development team for resolution.

4.6 Accessibility

The indicators will be designed to be accessible to users with disabilities, following WeAther-App's accessibility guidelines.

4.7 Technical Requirements

- **Image Format:** The indicators will use PNG images.
- **Image Size:** The images will be optimized for display on various devices and screen sizes.
- **Color Scheme:** The indicators will use a color scheme consistent with WeAther-App's brand guidelines.
- **Data Source:** The indicators will retrieve data from WeAther-App's weather API.

4.8 Testing and Validation

The indicators will undergo thorough testing and validation to ensure accuracy, reliability, and performance.

Testing will include:

- **Unit Testing:** Individual components will be tested to ensure correct functionality.
- **Integration Testing:** The indicators will be tested as part of the larger WeAther-App system.
- **User Acceptance Testing (UAT):** The indicators will be tested by users to ensure they meet requirements and are user-friendly.

5. Deployment and Maintenance

Chapter 5: Deployment and Maintenance

5.1 Overview

The WeAther-App deployment and maintenance process ensures the application's reliability, scalability, and performance. This chapter outlines the technical procedures for deploying and maintaining the WeAther-App.

5.2 Deployment Architecture

The WeAther-App deployment architecture consists of the following components:

- **Load Balancer (LB):** Distributes incoming traffic across multiple instances of the application.
- **Application Server (AS):** Hosts the WeAther-App and handles user requests.
- **Database Server (DS):** Stores and manages application data.
- **Cache Server (CS):** Stores frequently accessed data to improve performance.

5.3 Deployment Process

1. Pre-Deployment Checklist:

- Verify the application code is up-to-date and free of errors.
- Ensure all dependencies are installed and configured.
- Confirm the database schema is up-to-date.

2. Deployment Steps:

- Create a new instance of the Application Server (AS).
- Deploy the WeAther-App code to the AS instance.
- Configure the Load Balancer (LB) to route traffic to the new AS instance.
- Update the Database Server (DS) schema if necessary.
- Restart the Cache Server (CS) to refresh cached data.

3. Post-Deployment Verification:

- Verify the application is accessible and functional.
- Monitor application logs for errors.
- Perform load testing to ensure scalability.

5.4 Maintenance Procedures

1. Backup and Recovery:

- Schedule daily backups of the Database Server (DS) data.
- Store backups in a secure, off-site location.
- Test backup restoration procedures quarterly.

2. Security Updates:

- Monitor for security vulnerabilities in dependencies and the application code.
- Apply security patches and updates as needed.

3. Performance Optimization:

- Monitor application performance metrics (e.g., response time, CPU usage).
- Analyze performance bottlenecks and implement optimizations.

4. Error Handling:

- Implement error logging and notification mechanisms.
- Establish procedures for error investigation and resolution.

5.5 Monitoring and Logging

1. Monitoring Tools:

- Implement monitoring tools (e.g., Prometheus, Grafana) to track application performance and health.
- Configure alerts for critical issues (e.g., application crashes, database errors).

2. Logging Mechanisms:

- Implement logging mechanisms (e.g., Log4j, ELK Stack) to collect and store application logs.
- Configure log rotation and retention policies.

5.6 Update and Rollback Procedures

1. Update Procedures:

- Develop and test update scripts for the application code and dependencies.
- Schedule updates during maintenance windows.

2. Rollback Procedures:

- Develop and test rollback scripts for the application code and dependencies.
- Establish procedures for rolling back updates in case of issues.

5.7 Compliance and Security

1. Compliance Requirements:

- Ensure the WeAther-App complies with relevant regulations (e.g., GDPR, HIPAA).
- Implement compliance-related features (e.g., data encryption, access controls).

2. Security Best Practices:

- Implement security best practices (e.g., secure coding, secure data storage).
- Regularly review and update security procedures to ensure the application's security posture.

6. Testing and Quality Assurance

Chapter 6: Testing and Quality Assurance

6.1 Overview

WeAther-App testing and quality assurance processes ensure the application meets the required standards for functionality, performance, security, and usability.

6.2 Test Scope

The following components are subject to testing:

- User Interface (UI)
- Application Programming Interface (API)
- Data Storage and Retrieval
- Weather Forecasting Algorithm
- Integration with Third-Party Services
- Security Features

6.3 Test Types

The following test types are employed:

- **Unit Testing:** Verifies individual components function correctly
- **Integration Testing:** Verifies interactions between components
- **System Testing:** Verifies the entire application functions correctly
- **Acceptance Testing:** Verifies the application meets user requirements
- **Regression Testing:** Verifies changes do not introduce new defects
- **Security Testing:** Identifies vulnerabilities and ensures secure data handling

6.4 Test Cases

Test cases are designed to cover the following scenarios:

- Valid and invalid user input
- Normal and extreme weather conditions
- Network connectivity and disconnections
- Data storage and retrieval errors
- API request and response errors
- Security threats and vulnerabilities

6.5 Test Environment

The test environment consists of:

- **Hardware:** Multiple devices and platforms (e.g., Android, iOS, web)
- **Software:** Various operating systems, browsers, and API versions
- **Network:** Simulated network conditions (e.g., slow, fast, disconnected)
- **Data:** Sample data sets for testing (e.g., weather forecasts, user profiles)

6.6 Testing Tools

The following testing tools are employed:

- **JUnit:** Unit testing framework for Java
- **PyUnit:** Unit testing framework for Python
- **Selenium:** Automated UI testing framework
- **Postman:** API testing and debugging tool
- **OWASP ZAP:** Security testing and vulnerability scanning tool

6.7 Quality Assurance Metrics

The following metrics are used to measure testing effectiveness:

- **Test Coverage:** Percentage of code covered by tests
- **Test Pass Rate:** Percentage of tests passed
- **Defect Density:** Number of defects per unit of code
- **Mean Time to Detect (MTTD):** Time to detect defects
- **Mean Time to Resolve (MTTR):** Time to resolve defects

6.8 Defect Reporting and Tracking

Defects are reported and tracked using:

- **Defect Tracking System:** A centralized system for reporting and tracking defects
- **Defect Classification:** Defects are classified by severity and priority
- **Defect Resolution:** Defects are resolved and verified by the development team

6.9 Continuous Integration and Continuous Deployment (CI/CD)

The CI/CD pipeline automates:

- **Code Review:** Code is reviewed for quality and security
- **Automated Testing:** Tests are run automatically on code changes
- **Deployment:** Code is deployed to production after successful testing
- **Monitoring:** Application performance and security are continuously monitored

Appendix: Module Dependency Graph

